STRATA / CAREER INTELLIGENCE PLATFORM

Software Survey

A pre-build assessment of the dashboard's utilities, component requirements, constraints, and the risks we should plan around before committing to engineering work.

Project	cv_analyser (STRATA frontend + Flask backend)
Repository	d:/Projects/Portfolio/cv_analyser
Survey author	sagar (sagarvincnt@gmail.com)
Date	2026-05-19
Current state	React hi-fi prototype + Flask scaffold with stub scoring
Survey scope	Functions to ship, requirements, constraints, risks

---

1. Dashboard functions & utilities

Before discussing requirements, this section enumerates every function the STRATA dashboard intends to offer the end user. The list is derived from the design handoff (design_handoff_strata/README.md), the built React modules under frontend/src/components, and the central mock store at frontend/src/data/mockData.js. Each item is a discrete capability that the production system must back with real logic and data.

1.1 Intake & ingestion

- Upload a CV file (PDF / DOC / DOCX / TXT) via drag-and-drop or file picker.
- Optional paste of a target Job Description (JD) as free text.
- Validate file type, file name, and a 16 MB upload cap.
- 'Load sample profile (Aria Chen)' button to populate a canned demo profile + JD.
- Render a 12-stage live 'reasoning trace' animation (INGEST -> EMBED -> MATCH -> MARKET -> FIT -> GAP -> PATHS -> SYNTHESIS) while the real pipeline runs in the background.

1.2 Overview lens (executive brief)

- Executive headline + one-paragraph synthesis of all eight analytical lenses.
- Six clickable metric cards (JD Fit, ATS Score, Peer Percentile, Comp Position, Market Align, Momentum) - each navigates to its own lens.
- 'The three moves' - the three highest-leverage, ranked recommendations with expected delta tags.
- Live market mini-card (open roles, applicants/role + 7-point sparklines).
- Vector fingerprint card showing 20-of-1536-d embedding preview + cohort tag.

1.3 JD Fit lens

- Overall JD<->CV alignment score (gauge ring, 0-100).
- Counts of evidenced matches vs. JD requirements and gaps flagged (high / med / low).
- Keyword density vs. JD threshold.
- Evidence stream: top semantic matches with the exact CV phrase that grounds each match.
- Gap stream: ranked missing requirements with HIGH/MED/LOW impact pills + remediation notes.

1.4 ATS Readability lens

- ATS score (0-100) with a large ring visual.
- Eight structural checks: parseable structure, standard section headers, no images / multi-column layout, job-title taxonomy match (ESCO), keyword density, action-verb leads, quantified outcomes, skill-section taxonomy match.

- Per-check PASS / FIX verdict with explanatory notes.
- Recruiter dwell-time estimate vs. ATS-corpus norm.

1.5 Skill Matrix lens

- 8 x 5 heatmap of skill strength vs. four reference tracks (YOU / PEER P50 / PEER P90 / JD ASK / DIRECTOR TIER).
- Delta cards: biggest over-index, biggest under-index, most-undervalued skill.
- Switchable cell rendering (blocks vs. dots) via the chart-style power-user tweak.

1.6 Peer Benchmark lens

- 16-bucket histogram of overall peer-percentile distribution; user bucket and P50 bucket highlighted.
- Sub-dimension strip: tenure, mobility, impact, breadth - YOU vs. P50 with deltas.
- Cohort size disclosed (n = ~12,400).

1.7 Compensation lens

- Comp-band visual: min / p25 / p50 / p75 / max range with the user's current pay marked.
- Reference cards: market median, target band for next role, equity weight vs. cohort.
- Cohort definition (role, geo, industry, vintage of source offers).

1.8 Alternative Paths lens

- Career topology graph (SVG): centre node = current role; ring 1 = one-hop adjacent roles (n=5); ring 2 = two-hop reachable roles (n=5).
- Each node: title, fit score, salary, parent edge.
- Edge thickness and node colour encoded by fit.
- Two render modes (dots / blocks) with per-lens default.

1.9 Market Trends lens

- 'Rising' and 'Falling' lists of role families / skills with 6-month rolling demand deltas and 7-point sparklines.
- Editorial insight under each list (synthesis copy).

1.10 Market Alignment lens

- Six-axis radar (Demand / Comp / Mobility / Stability / Growth / Scarcity) overlaying the user's profile shape against market direction.
- Per-axis delta cards with signed +N / -N values, colour-coded by direction.

1.11 Profile, security & data settings

- Account tab: read-only-then-editable identity fields, last-updated stamp, save.
- Security tab: change-password flow with strength meter and validation rules (>=10 chars, confirm match), 2FA management, recovery-code regeneration, active session list with revoke.
- Data & Privacy tab: data-on-file counters, export-as-JSON, export-analyses-as-PDF, purge-all, analytics/research toggles.
- Danger Zone tab: deactivate (reversible 30 d) and permanent delete with type-DELETE confirmation + scheduled-deletion view and undo.

1.12 Chrome utilities (cross-cutting)

- Top-bar Export-PDF (server-rendered active lens + Overview brief).
- Top-bar Share-Brief (shareable link to a read-only snapshot).
- Sidebar 'NEW ANALYSIS' button (restart flow).

- Theme switch (dark default / light) and density (compact / regular / comfy).
- Tweaks panel exists as design-time tooling only; strip from production.

2. What should be built

The frontend prototype is essentially complete and matches the design spec one-to-one. The production system on the backend, however, is a thin stub: scoring_fun.score.score_cv returns a hard-coded 0 and the literal string 'Sample explanation', and similarity_score.sim_score is a lemma-overlap ratio over spaCy tokens with stopword removal. None of the eight lenses surfaced on the dashboard are computed by real logic; all numbers visible in the UI come from frontend/src/data/mockData.js. The build, therefore, is the entire analytical and data backend behind the dashboard.

2.1 Build targets (in priority order)

01. CV ingestion service: robust PDF / DOCX / DOC / TXT parser that yields a structured resume object (employers, dates, titles, bullets, skills, education, contact).
02. JD ingestion service: light NLP over pasted text to extract seniority, domain, scope, required skills, and keyword set.
03. Embedding service: produce a fixed-dimension career-space vector per CV and per JD (the prototype labels it strata-emb-3 / 1536-d; pick a real model).
04. Eight lens analyzers, each returning the exact data shape the React module already consumes (see Section 3 for the contract per lens).
05. Peer + market + comp reference datasets and the retrieval layer to query them.
06. Analysis orchestrator: fan-out across the lenses, stream stage events to the frontend's 12-stage trace, return one consolidated brief.
07. Persistence: users, CVs (with retention rules), analyses, embeddings cache.
08. Auth + billing: account/security/data tabs already assume Pro plan, 2FA, sessions, and delete-account flows.
09. PDF brief export (server-rendered Overview + active lens).
10. Observability: trace IDs (already shown in the UI - e.g. 0x4A7F2), latency / quality metrics, model-version logging.

2.2 Explicit non-goals

- Multi-tenant team workspaces - not in the design.
- Inline CV editing inside STRATA - the product critiques but does not author.
- Mobile-native apps - the design is desktop-class dark UI.
- Replacing the Tweaks panel with a production settings screen - it's a dev tool only.

3. Requirements by component / service

Requirements are organised by the service boundaries that should fall out of the build-targets above. Where the prototype already declares an interface (typed data shape, mock structure, UI affordance), the production component must honour it.

3.1 CV Ingestion Service

- Accept PDF, DOCX, DOC, TXT up to 16 MB; reject anything else with a clear error.
- Extract: contact block, summary, employers[], titles[], date-ranges[], bullets[], skills[], education[], certifications[], links[].
- Normalise titles to a canonical taxonomy (ESCO is named in the ATS spec).
- Defensive parsing of multi-column / scanned PDFs (fall back to OCR or flag).
- Strip secrets / PII for analytics; keep originals only for the user's retention window.
- Idempotent: same file content -> same parsed object (hash-keyed).

3.2 JD Ingestion Service

- Tokenise + classify: extract seniority, domain, scope, function, must-haves, nice-to-haves.
- Produce a normalised skill set keyed to the same taxonomy as the CV service.
- Handle pasted text from any ATS (LinkedIn / Greenhouse / Lever / freeform).

3.3 Embedding Service

- Single source of truth for vector dimension; the UI displays a 1536-d preview.
- Stable model version exposed in responses (the prototype tags 'strata-v3').
- Cache by content hash; recompute only on model version bump.
- Latency budget: the 12-stage trace targets a 14 s median total analysis (per the upload screen's capability strip).

3.4 Lens analyzers

Each lens is a function from (parsed_cv, parsed_jd, reference_data) to the exact structure rendered by its React module. Contracts:

JD Fit	score 0-100; evidencedMatches; totalRequirements; gapsFlagged (h/m/l breakdown); keywordDensity; matches[] of (topic, evidence quote, strength 0-1); gaps[] of (topic, impact, note).
ATS	score 0-100; dwellTime / normDwellTime; checks[] of (label, pass: bool, note). Must include all 8 checks in mockData.js verbatim.
Skill Matrix	rows[] (skills), cols[] (tracks), data[rows][cols] in 0..1, deltaCards[] for over-index / under-index / undervalued.
Peer Benchmark	buckets[16] of histogram counts, youBucket index, p50Bucket index, dimensions[] (dim, you, p50). Must publish cohort size + definition.
Compensation	min / p25 / p50 / p75 / max / you (all integers, USD); refCards[] for median, target band, equity weight. Cohort definition + sample size mandatory.
Alt Paths	center node + nodes[] (id, title, fit 0-1, salary string, ring 1|2) + links[] (from, to, fit). 10 nodes total in the mock; production may vary.
Trends	rising[] and falling[] of (topic, delta string %, spark[7], note).
Align	axes[6], you[6], market[6] - all values 0..1.
Overview	Synthesises the above into 6 metric cards + 3 ranked moves + market mini + vector fingerprint. Must not duplicate compute - it reads from lens outputs.

3.5 Reference data layer

- Peer profile corpus with cohort filters (role, seniority, geo, vintage).
- Live posting feed (the upload screen advertises 180K live postings, 320 markets).
- Verified-offer comp dataset (compSummary advertises n=1,840 over 90 d).
- Skill / title / industry taxonomy with synonyms.
- Refresh cadence and provenance per source - exposed in UI footnotes.

3.6 Orchestrator + streaming trace

- Server-Sent Events (or WebSocket) channel that streams the 12 trace messages by stage tag.
- If real compute returns early, hold on the final stage briefly; if late, the scan-line must keep moving (per design spec section 5.2).
- Trace ID format already shown: 0xNNNNN (5 hex chars).

3.7 Persistence

- Users, plans (Strata Pro mentioned), 2FA secrets, recovery codes, active sessions.
- CVs with explicit retention (the Data & Privacy tab advertises 'auto-purged in 28d', '30d / 12mo / 7d anonymisation').
- Analyses (the profile shows ANALYSES RUN counter).
- Embeddings cache (the profile shows VECTORS CACHED counter).

3.8 Auth, billing, account management

- OAuth (Google / Apple) + email magic link as a baseline.
- Stripe (or equivalent) for the Pro plan + renewal date displayed in profile stats.
- TOTP-based 2FA, recovery codes, device-session list with revoke.
- Permanent-delete pipeline: type-DELETE confirm, 30 d scheduled purge, undo before the scheduled date
- all already designed in the Danger Zone tab.

3.9 PDF / Share export

- Server-side render of the Overview brief plus the currently active lens.
- Headless-browser or React-PDF; must reproduce the OKLCH palette and serif headlines.
- Share-Brief: signed, read-only public URL with optional expiry.

3.10 Frontend (already in place, gaps to close)

- Replace MODULES' mock imports with real fetches against the new API.
- Add a router (the current App.jsx uses local state; spec recommends URL routes /upload, /analysing, /dashboard/[lensId], /profile/[tab]).
- Add a11y wiring (focus rings, aria labels on SVG, keyboard traversal of the lens list).
- Strip the TweaksPanel for production builds.

4. Constraints

4.1 Technical constraints (already locked)

- Frontend stack: React 18 + Vite (see frontend/package.json). The design's Next.js recommendation is not what we built.
- Backend stack: Flask 3 + spaCy en_core_web_sm; PyPDF2 / pdfplumber / python-docx / docx2txt for parsing (requirements.txt).
- Single-server deployment topology today (Dockerfile + docker-compose.yaml + an ECS task definition under .github/workflows/).
- 16 MB hard upload cap, file extensions limited to PDF/DOC/DOCX/TXT (interface.py).
- spaCy small English model only - no domain vocabulary, no multi-lingual support.
- Frontend is bundled and served from frontend/dist by Flask - no separate CDN today.

4.2 Design constraints

- High-fidelity visual reproduction is required: exact OKLCH tokens, Instrument Serif / Space Grotesk / JetBrains Mono fonts, density variants, animation easings.
- Editorial voice is part of the brand - terse, confident, numeric. The model output MUST be post-processed into this register, not surfaced raw.
- Glow is reserved for accent / good tones only. Never glow warn / bad / muted.
- One chart per card (except Skill Matrix).

4.3 Product constraints

- Single-user product - no team / org tier in scope.
- Desktop-first dark UI; mobile is out of scope until later.
- PDF / DOCX / DOC / TXT input only; no LinkedIn-import scraping path defined.
- English-only copy and analysis in v1 (the lemma overlap and the spaCy model assume English).

4.4 Compliance / legal constraints

- CV uploads contain PII. Retention windows are already promised in the UI ('auto-purged in 28d', 30d / 12mo / 7d anonymisation) - those promises become contractual once shipped.
- Peer + comp data sources determine the compliance posture: scraped data has different exposure than opt-in or partner data.
- Permanent-delete must actually delete (incl. derived vectors and analyses) within the advertised 30 d window.
- If analytics opt-out is OFF by default for benchmark contribution (as designed), the opposite default would breach the displayed copy.

4.5 Operational constraints

- 14 s median analysis target advertised on the upload screen - this becomes the end-to-end p50 latency budget for the full lens fan-out.
- The 12-stage trace must keep moving even if real compute stalls; UI cannot freeze.
- Versioning: the chrome already shows 'STRATA v4.2.1' and a trace ID - build/version headers must be wired through from CI.

5. Risks & potential problems

5.1 Backend correctness & accuracy

- Current scorer is a stub (scoring_fun.py returns 0 + 'Sample explanation') and the lemma-overlap fallback in similarity_score.py is a poor proxy for semantic fit. Shipping the existing UI on top of these will mislead users.
- Lemma overlap ignores synonyms, multi-word skills, and weighting - 'Python' and 'python developer' are not equivalent under the current code.
- Numbers displayed in the UI today (84p, 91/100, $214k, etc.) are mock and must be computed; there is no plan-of-record for any of them in the repo.
- Risk that the 'evidence quote' in JD Fit can be hallucinated unless we ground each match to a verbatim CV span.

5.2 Data sourcing & licensing

- No peer / market / compensation dataset exists in the repo. Acquiring one is the single largest unscoped expense (Levels.fyi / Pave / Compete / partner ATS feeds).
- Scraping LinkedIn or job boards risks ToS / legal exposure.
- Opt-in benchmark contribution (designed for the Data & Privacy toggles) seeds the peer pool, but starts empty - cold-start problem.

5.3 PII, security & privacy

- CVs leak names, emails, phone, addresses, employers. Encryption at rest + in transit is mandatory.
- The current Flask app uses a default dev secret if SECRET_KEY isn't set (interface.py:21) - dangerous if shipped that way.
- Permanent-delete must cascade to embeddings + cached derivations or the promise is broken.
- No rate limiting visible today - upload endpoint is open to abuse.
- The 'secure_filename' helper does not prevent malicious content inside accepted extensions (PDF parser RCE / docx zip-slip).

5.4 Parsing fragility

- PDF parsing of scanned / image-only CVs returns empty text - no OCR path today.
- Multi-column layouts produce interleaved text from PyPDF2.
- DOCX with embedded images / tables can break docx2txt extraction.
- parse_cv_file's first branch in parsing_fun.py checks isinstance(file_path, PdfWriter) but the type hint says str - dead code path, but a smell that the API isn't settled.

5.5 Latency & cost

- 14 s p50 target with embedding + peer retrieval + 8 lens analyzers is tight. Concurrent fan-out + caching is required, not optional.
- If embeddings come from a paid API, per-analysis cost compounds with daily users; vector cache is a must.
- Each lens has its own DB query / data source; without an orchestration layer, tail latency dominates.

5.6 UI / UX risks

- Dashboard renders 9 charts in real time on a single page - heavy DOM + SVG. Lens components re-mount on switch (by design), so fade-ins re-trigger - watch for layout thrash on slower hardware.
- Glow shadows and OKLCH gradients perform differently across Safari / Firefox / Chromium; visual QA on all three is needed.
- No accessibility wiring today - focus rings and aria roles are explicitly out-of-spec and must be added before launch.

- Editorial copy is generated by the model; without human review or a strict tone prompt, output drifts off-brand.

5.7 Auth / billing complexity

- Profile page assumes a finished auth system: 2FA, recovery codes, sessions, subscription plans, deletion queues. None of these exist in the backend today.
- Stripe integration + plan-state syncing introduces a separate consistency problem (webhooks, retries, dunning).

5.8 Product / scope risks

- The product makes nine independent analytical claims per analysis. Each weak claim erodes trust in the others; quality bar must be uniform.
- Output is advisory but the tone is decisive ('You're underpaid by $28k.'). Mis-calibrated numbers create reputational risk.
- Scope creep is highly likely - 'Director path' calculations alone could expand into a multi-month research project. Sequencing matters.

5.9 Repo hygiene & deployment

- Hardcoded local Windows paths still appear in parsing_fun.py and similarity_score.py __main__ blocks - harmless but noisy.
- Compiled .pyc files in backend/__pycache__ are tracked / modified - .gitignore exists but the cache files were already committed previously.
- compose.yaml was deleted in working tree; docker-compose.yaml is new (untracked). Pick one and commit it before CI / ECS workflow drift sets in.
- ECS deployment workflow (.github/workflows/ecs-deployment.yaml) implies cloud infra that the dev environment doesn't reflect.

6. Recommended build sequence

01. Backend foundation: replace the stub scorer with a deterministic JD Fit + ATS pipeline. These two lenses don't require external data and unblock honest dashboard demos.
02. Embedding + caching layer: pick a model, version it, cache by content hash.
03. Peer / comp / market data acquisition decision (build, partner, or punt). Block Skill Matrix, Peer, Comp, Trends, Align lenses on this call.
04. Orchestrator + streaming trace: make the 12-stage trace driven by real backend events.
05. Auth + persistence + plan management.
06. Profile / data / danger zone flows (mostly UI already exists; backend behind them is new).
07. PDF export + Share-Brief.
08. Accessibility + cross-browser pass + observability + cost dashboards.

7. Open questions to resolve before kickoff

01. CV parsing: roll our own pipeline or adopt Affinda / Docparser / RChilli?
02. Embedding model: OpenAI text-embedding-3-large (1536-d, matches UI), Cohere embed-v3, or a custom fine-tune?
03. Peer dataset provenance: opt-in only, partner ATS, scraped, or hybrid?
04. Compensation data: Levels.fyi-style scrape vs. partner data (Pave, Compete)?
05. Auth providers: Google + Apple + magic link sufficient, or do we need SSO?
06. Billing: Stripe Customer Portal vs. custom plan-management UI?

07. Production deployment target: stay on AWS ECS (per the existing workflow) or move to a managed PaaS?
08. Frontend hosting: continue serving frontend/dist from Flask, or split to a CDN + API?